



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий (ИТ)

Кафедра Математического обеспечения и стандартизации информационных технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 3

по дисциплине

«Технология разработки программных приложений»

Тема: «Системы сборки»

Выполнил:

студент группы ИКБО-20-23

Кузнецов Л. А.

Принял:

Доцент кафедры МОСИТ,
кандидат технических наук,
доцент
Чернов Е. А.

Практическая работа выполнена

«__»_____202__г.

(подпись студента)

«Зачтено»

«__»_____202__ г.

(подпись руководителя)

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ	3
2 ХОД РАБОТЫ	4
2.1 Образы	4
2.2 Изоляция	5
2.3 Работа с портами	6
2.4 Именованные контейнеры, остановка и удаление.....	8
2.5 Постоянное хранение данных.....	9
2.6 Переменные окружения.....	14
2.7 Dockerfile.....	14
2.8 Индивидуальные задания	16
3 ВЫВОД.....	18

1 ЦЕЛЬ РАБОТЫ

Знакомство с системой Docker

2 ХОД РАБОТЫ

2.1 Образы

Посмотрите на имеющиеся образы: `docker images`.

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker images
REPOSITORY    TAG       IMAGE ID   CREATED   SIZE
```

Загрузите образ: `docker pull ubuntu` — будет загружен образ `ubuntu:latest` — последняя доступная версия. Для загрузки конкретной версии, нужно указать тег, например, 12.04: `docker pull ubuntu:12.04`

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
5a7813e071bf: Download complete
Digest: sha256:72297848456d5d37d1262630108ab308d3e9ec7ed1c3286a32fe09856619a782
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker pull ubuntu:12.04
12.04: Pulling from library/ubuntu
6d93b41cfc6b: Download complete
83251ac64627: Download complete
589bba2f1b36: Download complete
d62ecaceda39: Download complete
d8868e50ac4c: Download complete
Digest: sha256:18305429afa14ea462f810146ba44d4363ae76e4c8dfc38288cf73aa07485005
Status: Downloaded newer image for ubuntu:12.04
docker.io/library/ubuntu:12.04
```

Посмотрите на имеющиеся образы ещё раз: `docker images` — должны появиться новые загруженные образы. Посмотрите список контейнеров, выполнив команду: `docker ps -a`

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker images
REPOSITORY    TAG       IMAGE ID   CREATED   SIZE
ubuntu        latest    72297848456d  7 weeks ago  117MB
ubuntu        12.04     18305429afa1  7 years ago  162MB

\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
```

2.2 Изоляция

Посмотрим информацию о хостовой системе, выполнив команду `hostname`. Выполните её ещё один раз.

Вопрос: одинаковый ли результат получился при разных запусках? (Да)

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>hostname
LAPTOP

\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>hostname
LAPTOP
```

Попробуем выполнить то же самое в контейнерах. Выполните два раза команду `docker run ubuntu hostname`.

Вопрос: Одинаковый ли результат получился при разных запусках? (Нет)

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run ubuntu hostname
c1187d6e5c16

\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run ubuntu hostname
9666158b26e3
```

В случае запуска команды в контейнерах, ответ будет немного отличаться, будет разный `hostname`. Так происходит, потому что из одного образа `ubuntu` были запущены два изолированных контейнера, поэтому у них и был разный `hostname`.

Заново выполните `docker ps -a` — там должны появиться запущенные ранее контейнеры.

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
9666158b26e3   ubuntu   "hostname"              48 seconds ago Exited (0) 47 seconds ago           peaceful_hermann
c1187d6e5c16   ubuntu   "hostname"              About a minute ago Exited (0) About a minute ago           elated_joliot
```

Запуск контейнеров производится командой: `docker run --флаги --докера образ` (а не имя контейнера как указано в методичке) команда для запуска -и -флаги --запуска --программы.

Запустите `bash` в контейнере: `docker run ubuntu bash`.

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run ubuntu bash
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>
```

Ничего не произошло. Это не баг. Интерактивные оболочки выйдут после выполнения любых скриптовых команд, если только они не будут запущены в интерактивном терминале — поэтому для того, чтобы этот пример не завершился, вам нужно добавить флаги `-i -t` или сгруппированно `-it`: `docker run -it ubuntu bash`.

Выполняя запуск контейнера, указывая образ `ubuntu`, неявно указывался образ `ubuntu:latest`. Следовательно, следующие команды равнозначны:

- `docker run ubuntu hostname`
- `docker run ubuntu:latest hostname`

Выполняя запуск контейнера, указывая образ `ubuntu`, неявно указывался образ `ubuntu:latest`. Следовательно, следующие команды равнозначны:

- `docker run ubuntu hostname`
- `docker run ubuntu:latest hostname`

2.3 Работа с портами

Для начала, загрузите образ `python` командой `docker pull python`.

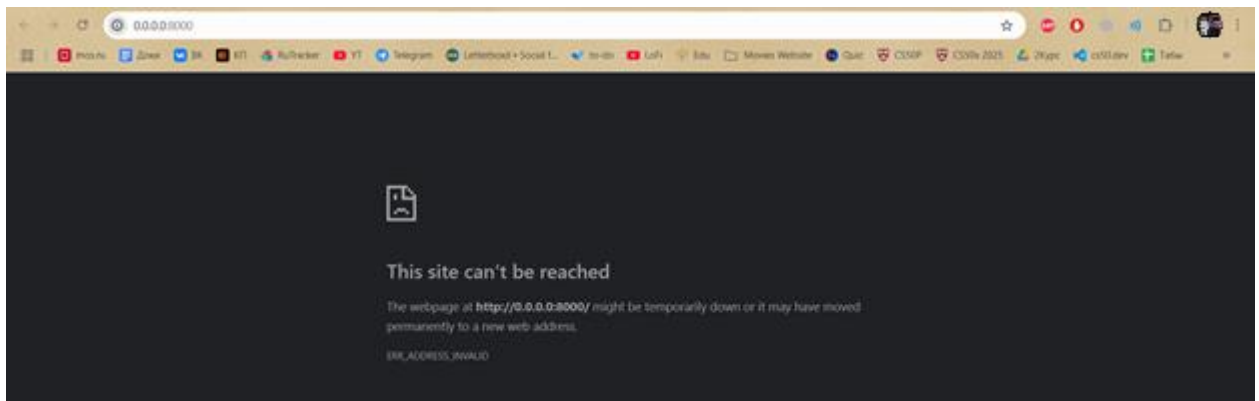
```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker pull python
Using default tag: latest
latest: Pulling from library/python
7cd785773db4: Download complete
70847531e23d: Download complete
af0b544b1a82: Download complete
379f06e37dbb: Download complete
091eb8249475: Download complete
255774e0027b: Download complete
353e14e5cc47: Download complete
Digest: sha256:8c55c44b9e81d537f8404d0000b7331863d134db87c1385dd0ec7fefff656495
Status: Downloaded newer image for python:latest
docker.io/library/python:latest

\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
python        latest    8c55c44b9e81   6 weeks ago    1.47GB
ubuntu        latest    72297848456d   7 weeks ago    117MB
ubuntu        12.04    18305429afa1   7 years ago    162MB
```

В качестве примера, запустите встроенный в Python модуль веб-сервера из корня контейнера, чтобы отобразить содержание контейнера.

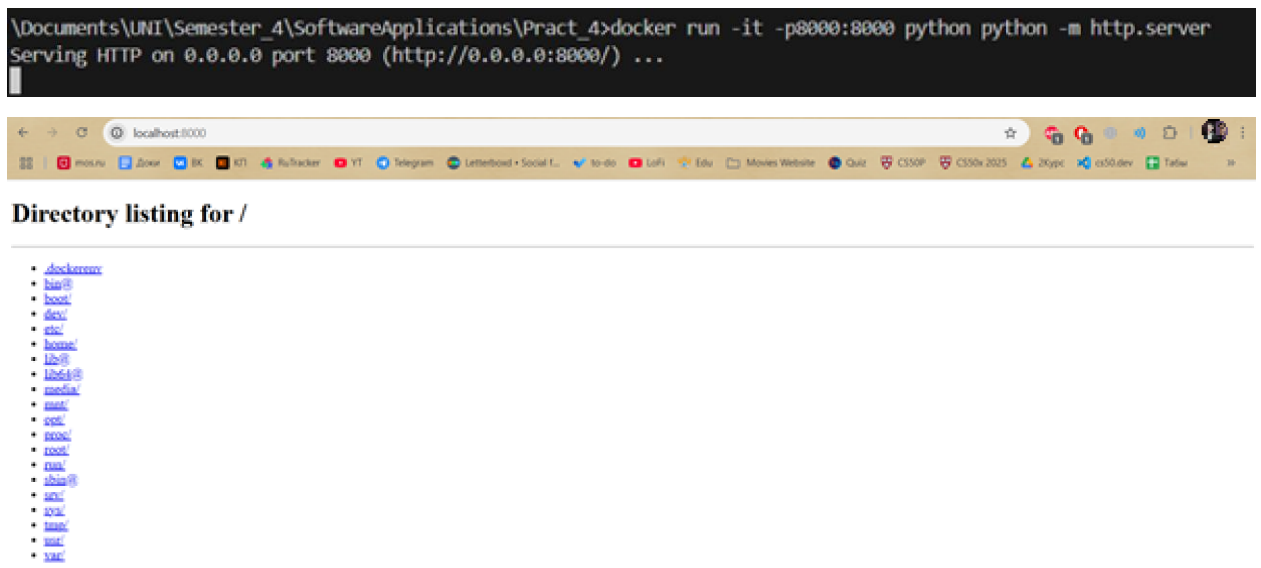
```
docker run -it python python -m http.server
```

При запуске пишется, что сервер доступен по адресу `http://0.0.0.0:8000/`. Однако, если открыть этот адрес, то ничего не будет видно, потому что порты не проброшены. Завершите работу веб-сервера, нажав комбинацию клавиш `Ctrl+C`.



Для проброса портов используется флаг `-p hostPort:containerPort`. Добавьте его, чтобы пробросить порт 8000:

`docker run -it -p8000:8000 python python -m http.server` — теперь по адресу `http://0.0.0.0: 8000/` (если не открывается на Windows, то вместо 0.0.0.0 нужно указать `localhost`) открывается содержимое корневой директории в контейнере.



Для того, чтобы доступный в контейнере на порту 8000 веб-сайт в хостовой системе открывался на порту 8888, необходимо указать флаг `-p 8888:8000`:

```
docker run -it -p8888:8000 python python -m http.server.
```

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run -it -p 8888:8000 python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Завершите работу веб-сервера, нажав комбинацию клавиш `Ctrl+C`.

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run -it -p 8888:8000 python python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
172.17.0.1 - - [19/Mar/2025 18:47:33] "GET / HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
```

2.4 Именованные контейнеры, остановка и удаление

Запустите контейнер: `docker run -it -p8000:8000 python python -m http.server`. Нажмите `Ctrl+C` — выполнение завершится. Для того, чтобы запустить контейнер в фоне, нужно добавить флаг `-d/--detach`. Также определим имя контейнеру, добавив флаг `--name`.

```
docker run -p8000:8000 --name pyserver -d python python -m http.server
```

Убедитесь, что контейнер всё ещё запущен: `docker ps | grep pyserver` — вывод команды не должен быть пустым. Для просмотра логов контейнера, воспользуйтесь командой `docker logs pyserver`

Для того, чтобы остановить выполнение контейнера, существует команда `docker stop pyserver`. Однако, если снова попробовать запустить командой `docker run -it -p8000:8000 --name pyserver -d python python -m http.server`, то возникнет ошибка: контейнер с таким именем существует. Его нужно удалить `docker rm pyserver`.

```
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker stop pyserver
pyserver
\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run -p8000:8000 --name pyserver -d python python -m http.server
docker: Error response from daemon: Conflict. The container name "/pyserver" is already in use by container "51611ccd553fc659b9d6857d
bb9cb92b3771e0814e3a72bb6e107e9267e155e6". You have to remove (or rename) that container to be able to reuse that name.
See 'docker run --help'.
```


Для остановки и удаления контейнера можно воспользоваться командой `docker rm -f pyserver` вместо выполнения двух отдельных команд `stop` и `rm`. После удаления контейнер с таким именем можно будет создать заново.

Для того, чтобы контейнер удалялся после завершения работы, нужно указать флаг `--rm` при его запуске — далее в работе мы будем использовать данный флаг: `docker run --rm -p8000:8000 --name pyserver -d python python -m http.server`

```
D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run --rm -p8000:8000 --name pyserver -d python python -m http.server
ff5b5cc39068486d839f3660f0889fb1b631a8c57750a0670e10698a0e50fc18

D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
ff5b5cc39068   python    "python -m http.serv..." 17 seconds ago Up 16 seconds 0.0.0.0:8000->8000/tcp             pyserver

D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker stop pyserver
pyserver
```

2.5 Постоянное хранение данных

Запустите контейнер, в котором веб-сервер будет отдавать содержимое директории `/mnt`: `docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt`, где `-d /mnt` указывает модулю `http.server` какая директория будет корневой для отображения.

Вопрос: Что значат остальные флаги запуска? Где здесь команда, которая выполнится в контейнере?

Ответ: Флаг `-p 8000:8000` пробрасывает порт 8000 с контейнера на хост, чтобы сервер был доступен извне.

Флаг `--rm` автоматически удаляет контейнер после его остановки. Флаг `-d` запускает контейнер в фоновом режиме.

Команда, которая выполнится внутри контейнера, — это `python -m http.server -d /mnt`, где `python` запускает интерпретатор, `-m http.server` активирует встроенный HTTP-сервер

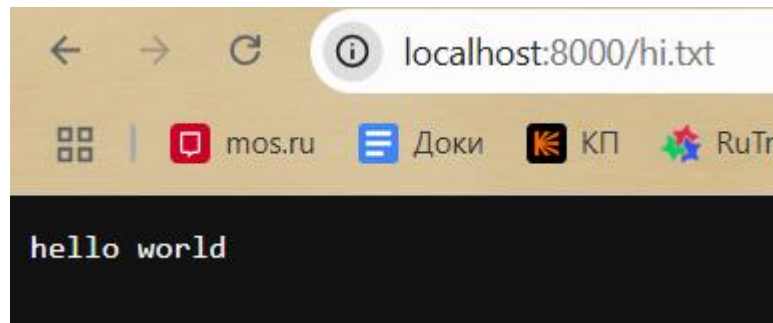
```
D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt
5f1f6eca188faf0acf09a4c421b7a5ae836353dd69f4147b01115d5991c8a85a
```

Для того, чтобы попасть в уже запущенный контейнер, существует команда `docker exec -it pyserver bash` — вы попадёте в оболочку `bash` в

контейнере. Попад в контейнер, выполните команду `cd mnt && echo "hello world" > hi.txt`, а затем выйдите из контейнера, введя команду `exit` или нажав комбинацию клавиш `Ctrl+D`.

```
D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker exec -it pyserver bash
root@5f1f6eca188f:/# cd mnt && echo "hello world" > hi.txt
root@5f1f6eca188f:/mnt# exit
exit
```

Если открыть `http://0.0.0.0:8000/`, там будет доступен файл `hi.txt`.



Остановим контейнер: `docker stop pyserver`, а затем снова запустим: `docker run -p8000:8000 --name pyserver --rm -d python python -m http.server -d /mnt`.



Error response

Error code: 404

Message: File not found.

Error code explanation: 404 - Nothing matches the given URI.

Как мы видим, файл `hi.txt` пропал — это неудивительно, ведь мы запустили другой контейнер, а старый был удалён после завершения работы (флаг `--rm`). Остановим контейнер: `docker stop pyserver`.

```
D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4>docker stop pyserver
pyserver
```

Для того, чтобы не терялись какие-то данные (например, если запущен контейнер с СУБД, то чтобы не терялись данные из неё) существует механизм монтирования.

2.5.1 Тома

Первый способ — это создать отдельный том с помощью ключа `-v myvolume:/mnt`, где `myvolume` — название тома, `/mnt` — директория в контейнере, где будут доступны данные. Попробуйте снова создать контейнер, но уже с примонтированным томом:

```
docker run -p8000 :8000 - - rm - - name pyserver - d \ - v $ ( pwd )/ myfiles
:/ mnt python python - m http . server - d / mnt
```

```
$ docker exec -it pyserver bash
root@c91aefaae3d0:/# cd mnt && echo "hello world" > hi.txt
root@c91aefaae3d0:/mnt# exit
exit
```

Затем, если создать файл (выполнить `docker exec -it pyserver bash` и внутри контейнера выполнить `cd mnt && echo "hello world" > hi.txt`), то даже после удаления контейнера данные в этом томе будут сохранены.

Чтобы узнать где хранятся данные, выполните команду `docker inspect -f "{{json .Mounts }}" pyserver`, в поле `Source` будет храниться путь до тома на хостовой машине

```
$ docker inspect -f "{{json .Mounts }}" pyserver
[{"Type":"bind","Source":"D:\\Boris\\Documents\\UNI\\Semester_4\\SoftwareApplications\\Pract_4\\myfiles;D",
"Destination":"\\Program Files\\Git\\mnt","Mode":"","RW":true,"Propagation":"rprivate"}]
```

Для управления томами существует команда `docker volume`, ознакомиться с которой предлагается самостоятельно.

```
$ docker volume --help

Usage:  docker volume COMMAND

Manage volumes

Commands:
  create      Create a volume
  inspect     Display detailed information on one or more volumes
  ls          List volumes
  prune       Remove unused local volumes
  rm          Remove one or more volumes

Run 'docker volume COMMAND --help' for more information on a command.
```

2.5.2 Монтирование директорий и файлов

Сперва, остановите контейнер, созданный на предыдущем шаге: `docker stop pyserver`.

```
$ docker stop pyserver
pyserver
```

Иногда требуется пробросить в контейнер конфигурационный файл или отдельную директорию. Для этого используется монтирование директорий и файлов.

Создадим директорию и файлы, которые будем монтировать. Часть из них нам понадобится дальше: создайте директорию: `mkdir myfiles`, в ней создайте файл `host.txt`: `touch myfiles/host.txt`

```
$ touch myfiles/host.txt
```

Запустите контейнер: `docker run -p8000:8000 --rm --name pyserver -d -v $(pwd)/myfiles:/mnt python \`
`> python -m http.server -d /mnt`

```
$ docker run -p8000:8000 --rm --name pyserver -d -v $(pwd)/myfiles:/mnt python \
> python -m http.server -d /mnt
b134a487bf6a166ca33ab7bbc29f7b9ebd8fa79be66ca0da8f06adeb83124342
```

Команда `pwd` — выведет текущую директорию, например: `/home/user/dome-directory`, в итоге получился абсолютный путь до файла: `/home/user/dome-directory/myfiles`. Обратный слеш (`\`) перед переводом строки

экранирует символ перевода строки и позволяет написать одну команду в несколько строк.

Затем, зайдите в контейнер: `docker exec -it pyserver bash`, перейдите в директорию `/mnt` командой `cd /mnt`. Если вывести список файлов командой `ls`, то там будет файл `host.txt`, примонтированный вместе с директорией `myfiles`

```
$ docker exec -it pyserver bash
root@c6ad2360693b:/#
```

```
root@c6ad2360693b:/# cd /mnt
root@c6ad2360693b:/mnt# ls
host.txt
```

Создайте файл `echo "hello world" > hi.txt`, а затем выйдите из контейнера: `exit`. Теперь на хостовой машине в директории `myfiles/` появится файл `hi.txt`. Проверить можно командой `ls myfiles`

```
$ docker exec -it pyserver bash
root@c6ad2360693b:/# cd /mnt
root@c6ad2360693b:/mnt# echo "hello world" > hi.txt
root@c6ad2360693b:/mnt# exit
exit
```

```
$ ls myfiles
hello.txt host.txt
```

Остановите контейнер: `docker stop pyserver`.

```
$ docker stop pyserver
pyserver
```

Для того, чтобы примонтировать один файл, нужно указать ключ `-v`, например:

`v $(pwd)/myfiles/host.txt:/mnt/new-name-of-host.txt` – файлу в контейнере присвоится другое имя: `new-name-of-host.txt`.

```
PS D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4> docker run -p 8000:8000 --rm --name pyserver -d -v
$(PWD)/myfiles/host.txt:/mnt/new-name-of-host.txt python http.server -d /mnt
6b357c2d13197624f61faa927ac4002ea6c9477b230462850fa3a2c059201d08
```

```
PS D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4> docker exec -it pyserver bash
root@6b357c2d1319:/# cd /mnt
root@6b357c2d1319:/mnt# ls
new-name-of-host.txt
```

2.6 Переменные окружения

Для передачи переменных окружения внутрь контейнера используется ключ `-e`. Например, чтобы передать в контейнер переменную окружения `MIREA` со значением «ONE LOVE», нужно добавить ключ `-e MIREA="ONE LOVE"`. Проверьте, выведя все переменные окружения, определённые в контейнере с помощью утилиты `env`: `docker run -it --rm -e MIREA="ONE LOVE" ubuntu env`. Среди списка переменных будет и `MIREA`

```
D:\Documents\UNI\Semester 4\SoftwareApplications\Pract 4>docker run -it --rm -e MIREA="ONE LOVE" ubuntu env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=017c869c16dc
TERM=xterm
MIREA=ONE LOVE
HOME=/root
```

2.7 Dockerfile

Соберите образ, в который будут установлены дополнительные пакеты, примонтируйте директорию и установите команду запуска. Для этого создаётся файл `Dockerfile` (без расширения).

```
1 FROM ubuntu:20.04
2 RUN apt update \
3     && apt install -y python3 fortune \
4     && cd /usr/bin \
5     && ln -s python3 python
6 RUN /usr/games/fortune > /mnt/greeting-while-building.txt
7 ADD ./data /mnt/data
8 EXPOSE 80
9 CMD ["python", "-m", "http.server", "-d", "/mnt/", "80"]
```

В строке (1) указывается базовый образ, на основе которого будет строиться новый образ.

В строках (2-5) указана команда, которая выполнится в процессе сборки. На самом деле, там выполняются несколько команд, соединённых `&&` для того, чтобы создавать меньше слоёв в образе.

В строках (6) тоже указана команда, которая сгенерирует случайную цитату и перенаправит вывод в файл `/mnt/greeting-while-building.txt`. Файл будет сгенерирован во время сборки образа.

В строке (7) копируется всё содержимое директории `./data` хостовой машины в директорию `/mnt`, которая будет доступна в контейнере.

В строке (8) указывается, какой порт у контейнера будет открыт.

В строке (9) указывается команда, которая будет выполнена при запуске, где 80 — порт, который будет слушать веб-сервер.

```
Dockerfile X
Dockerfile > ...
1 FROM ubuntu:20.04
2 RUN apt update \
3     && apt install -y python3 fortune \
4     && cd /usr/bin \
5     && ln -s python3 python
6 RUN /usr/games/fortune > /mnt/greeting-while-building.txt
7 ADD ./data /mnt/data
8 EXPOSE 80
9 CMD ["python", "-m", "http.server", "-d", "/mnt/", "80"]

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE OPEN EDITORS OUTLINE TIMELINE

PS D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4> code Dockerfile
PS D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4> 
```

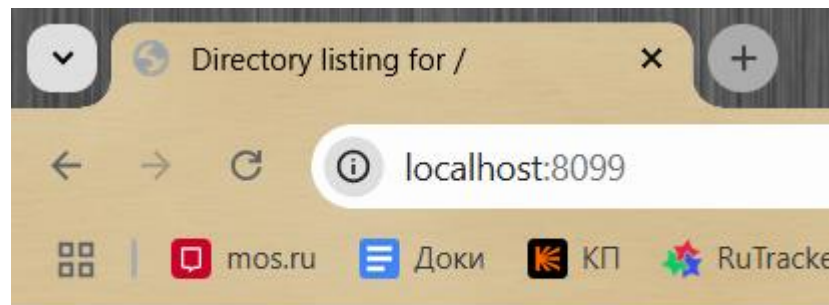
Соберите образ с тегом `myscoolimage` с помощью команды `docker build -t myscoolimage .` Точка в конце указывает на текущую директорию, где лежит `Dockerfile`.

```
[*] Building 47.5s (9/9) FINISHED
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 316B
-> [internal] load metadata for docker.io/library/ubuntu:20.04
-> [internal] load .dockerignore
-> => transferring context: 2B
-> [1/4] FROM docker.io/library/ubuntu:20.04@sha256:8e5caf0285ecb8ead070435d296576a530d1166df73ec4affc1cd27555141b
-> => resolve docker.io/library/ubuntu:20.04@sha256:8e5caf0285ecb8ead070435d296576a530d1166df73ec4affc1cd27555141b
-> => sha256:d9802f032dc798c2086607424bf681b8c1d6f116e11cd99592dcaf261e9cd2 27.519B / 27.519B
-> => extracting sha256:d9802f032dc798c2086607424bf681b8c1d6f116e11cd99592dcaf261e9cd2
-> [internal] load build context
-> => transferring context: 26B
-> [2/4] RUN apt update && apt install -y python3 fortune && cd /usr/bin && ln -s python3 python
-> [3/4] RUN /usr/games/fortune > /mnt/greeting-while-building.txt
-> [4/4] ADD ./data /mnt/data
-> => exporting image
-> => exporting layers
-> => exporting manifest sha256:002145b23fba3583657165b11e38e2add13f7dab36f7a145151182d809ee7a
-> => exporting config sha256:377d5af09ef2a2ed76c1aa2b27eb1999dc1f7661012e081c1f1661f002b
-> => exporting attestation manifest sha256:01b9e0721eab19f7eb0b867c915d8f0a8d7b847af66bc7ccc098ac1f09fda2
-> => exporting manifest list sha256:e07f266611e638bf3c02c3d69461a28b65719ef144c7332114fcb653defa73
-> => naming to docker.io/library/myscoolimage:latest
-> => unpacking to docker.io/library/myscoolimage:latest

View build details: docker-desktop:///dashboard/build/desktop-linux/desktop-linux/utatr00cd6c9a227rev99fe
PS D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4> 
```

Запуск производится командой `docker run --rm -it -p8099:80 myscoolimage`, где порт 8099 — порт на хостовой машине.

```
PS D:\Documents\UNI\Semester_4\SoftwareApplications\Pract_4> docker run --rm -it -p8099:80 mycoolimage
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
172.17.0.1 - - [20/Mar/2025 07:56:01] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [20/Mar/2025 07:56:01] code 404, message File not found
172.17.0.1 - - [20/Mar/2025 07:56:01] "GET /favicon.ico HTTP/1.1" 404 -
```



Directory listing for /

- [data/](#)
- [greeting-while-building.txt](#)

2.8 Индивидуальные задания

Номер варианта: 1

Написать Dockerfile, собрать образ, запустить контейнер (и записать команду для его запуска).

Для монтирования создайте директорию data и в ней файл student.txt, содержащий ФИО, название группы и номер варианта.

Для установки пакетов использовать команду `apt install -y название-пакета`. В качестве примера можно использовать Dockerfile из раздела 7.

Нечётные варианты:

- необходимо использовать базовый образ `ubuntu:20.04`
- примонтировать директорию data в директорию `/mnt/files/` в контейнере

3 ВЫВОД

В ходе выполнения практической работы были приобретены и закреплены навыки работы с Docker, включая создание и настройку контейнеров, работу с HTTP-сервером в контейнеризованной среде, а также сборку проекта с использованием Dockerfile. Освоенные технологии позволяют эффективно разворачивать и управлять приложениями в изолированных средах, обеспечивая их портативность и удобство в развертывании.